FUNDAMENTOS BÁSICOS

DO GIT & GITHUB

Guia para iniciantes na área de TI



Sumário

1. **Por que versionar código?**
   1.1. O que é o Git?

2. **Preparação do ambiente**
   2.1. Instalação (Windows)
   2.2. Configurações iniciais: git config

3. **Github**
   3.1. Criando uma conta no Github
   3.2. Outras formas de usar o Github

4. **O coração do Git**
   4.1. Entendendo o estado dos arquivos: Untracked, Modified, Staged e Committed

5. **Apresentação dos códigos principais para o fluxo do projeto**
   5.1 git init: Criando seu primeiro repositório
   5.2 git status: Verificando sua posição atual no projeto
   5.3 git add: Selecionando as mudanças
   5.4 git commit: Criando pontos na história (Boas práticas de mensagens)
   5.5 git log: Navegando pelo passado do projeto

6. **Conexão com o Github**
   6.1. git remote: Conectando o PC ao servidor
   6.2. git push: Enviando seu trabalho
   6.3. git clone: Baixando projetos existentes
   6.4. git pull: Mantendo seu código local atualizado

7. **Conteúdo Complementar: Branches**
   7.1. O que são ramificações e por que usá-las
   7.2. git branch e git checkout: Criando novas funcionalidades sem quebrar o código principal
   7.3. Conceito de Merge e Pull Request

8. **Se interessou no assunto?**
   9.1. Indicações de vídeos, cursos e ferramentas visuais

9. **Tabela de comandos e abreviações**
   8.1. Lista de comandos para consulta
   8.2. Sugestões de atalhos para ganhar produtividade no terminal

1. Por que versionar código?

Versionar não é apenas "salvar" o seu trabalho; é criar uma linhagem histórica de todas as decisões tomadas durante o desenvolvimento. Em vez de ter uma foto estática do seu código hoje, você tem um filme completo que mostra como ele era ontem, semana passada ou há um ano.

O controle de versão funciona como uma rede de segurança e uma máquina do tempo. Ele permite que desenvolvedores experimentem novas ideias, cometam erros e explorem caminhos diferentes sem o medo constante de destruir o que já foi construído. Sem o versionamento, a programação profissional seria como caminhar em uma corda bamba sem nenhum equipamento de proteção: qualquer deslize poderia ser fatal para o projeto.

Quando somos introduzidos na área, é comum acharmos que estamos seguros salvando arquivos na pasta em um caminho qualquer. Mas o caos chega rápido.

O erro do "Desfazer": Você apaga uma função, salva o arquivo, fecha o VS Code e percebe que precisava daquela função. *O Ctrl + Z* não vai te salvar.

O caos das versões: Você acaba com arquivos como *projeto_v1.c*, *projeto_v1.c*... Não concorda que isso não é nem um pouco prático?

O risco do hardware: Se o seu notebook pifar hoje, você perde o semestre? Se a resposta for sim, você precisa de versionamento.

Então diante de todos esses problemas, surge a necessidade do uso do Git. Mas o que é ele?

1.1. O que é o Git?

O Git é um Sistema de Controle de Versão Distribuído. Possuindo características que moldam o que entendemos sobre controle de código.

Snapshot, não cópia: O Git não copia o arquivo inteiro toda hora; ele tira uma "foto" (snapshot) do que mudou.

Máquina do Tempo: Ele permite que você navegue entre essas fotos. Se o código quebrou às 15h, você pode voltar para a versão exata das 14h com um comando.

Independência: Cada desenvolvedor tem uma cópia completa do histórico no seu próprio PC.

Avaliando essas 3 principais funcionalidades do Git, conseguimos entender muito bem a sua importância para nós desenvolvedores. Por isso que, para estudantes como vocês, o GitHub torna-se o seu currículo vivo. Ele pode proporcionar os seguintes aspectos:

Vitrine: Recrutadores olham o seu GitHub antes mesmo de lerem seu currículo. Eles querem ver se você sabe usar Git, se seu código é limpo e se você é consistente. Eles procuram saber se você é alguém que gosta de colocar em prática tudo aquilo que você desenvolveu por meio de projetos.

Colaboração: É lá que grandes projetos (como o próprio Linux ou o VS Code) são construídos. Aprender GitHub é aprender a trabalhar em equipe em escala global.

2. Preparação do Ambiente

2.1. Instalação (Windows)

A ferramenta padrão para Windows é o **Git for Windows**.

1. Acesse git-scm.com e baixe a versão estável.
2. Após instalar, abra o menu Iniciar, digite "Git Bash" e dê Enter para testar.

Tutorial de instalação: Como Instalar o GIT no Windows

Importante: Durante a instalação, certifique-se de que a opção **"Git Bash"** será instalada. Ele emula um terminal Linux no Windows, que é muito mais poderoso e o padrão na área de tecnologia.

2.2. Configurações iniciais: git config

O Git precisa "assinar" cada alteração que você faz. Para que tenhamos anotado quem realizou certas modificações no projeto. Se você não configurar sua identidade, ele não deixará você salvar nada.

No seu terminal (Git Bash), execute os comandos abaixo substituindo pelos seus dados reais:

```
# Define o seu nome (será exibido no histórico de commits)
git config --global user.name "Seu Nome Completo"

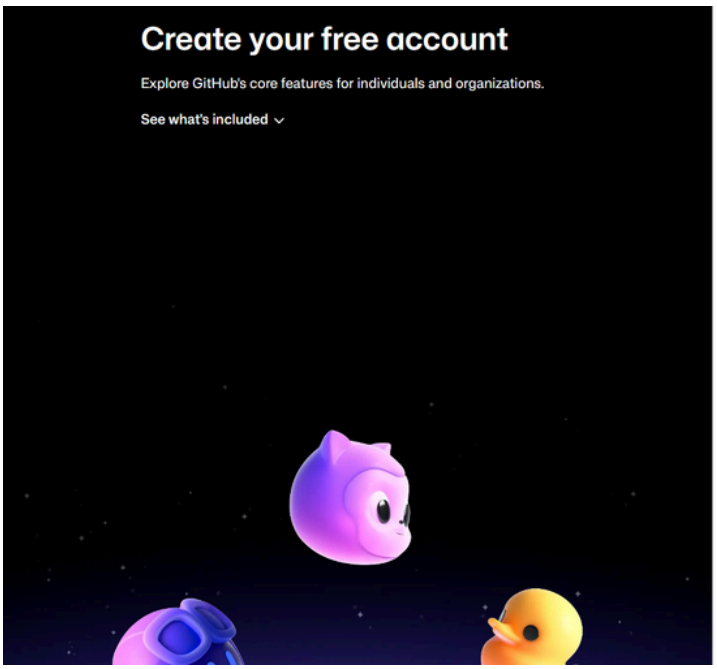
# Define o seu e-mail (DEVE ser o mesmo e-mail que você usará no GitHub)
git config --global user.email "seu_email@exemplo.com"

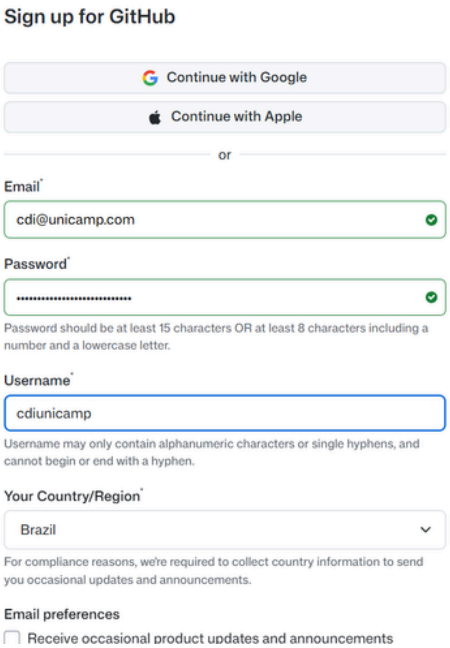
# Verifica se as configurações foram salvas corretamente
git config --list
```

3. GitHub

3.1. Criando uma conta no GitHub

Acesse github.com e efetue seu cadastro na seguinte página.





Dica da CDI: Escolha um nome de usuário (username) limpo e profissional. Evite apelidos de jogos. Use algo como `gabriel-sobrenome` ou `gabs-si`. Isso facilita ser encontrado por empresas.

3.2. Outras formas de usar o Github

Particularmente, brincar com o git no terminal é um tanto quanto prático e melhor, não precisando de instalação de ferramentas adicionais. Porém, embora o foco da apostila seja o uso do **Terminal** (porque é onde o desenvolvedor tem mais controle), existem outras interfaces que permitem o versionamento de código com o Git:

Github Desktop: Um programa visual para quem prefere não usar comandos.

> **Tutorial de instalação:** [Como instalar o GitHub Desktop?]

GitHub Mobile: Ótimo para revisar o código de colegas ou responder comentários de projetos pelo celular. Basta só acessar o no loja de aplicativos do seu celular e instalá-lo.

4. O coração do git

Para dominar o Git, você não precisa decorar comandos, você precisa entender onde o seu arquivo está. Diferente de um "Save" comum de Word, o Git possui um ciclo de vida, ou seja, trabalha com três ambientes distintos antes de uma alteração ser oficialmente salva. Imagine que você está organizando uma mudança de casa. O fluxo do Git funciona de forma muito parecida:

Diretório de Trabalho (Working Directory): É a sua pasta no computador. Onde você abre o VS Code e edita seu código. No mundo da mudança, é a sua sala com tudo espalhado.

Área de Preparação (Staging Area): Aqui você coloca os arquivos que já terminou de editar e que pretende salvar. É como se fosse a caixa da mudança: você coloca as coisas dentro, mas ainda não fechou a fita nem colocou no caminhão.

Repositório Local (Local Repository): É onde o Git guarda o histórico oficial. Quando você salva as suas alterações, você fecha a caixa e guarda no caminhão. Agora aquilo faz parte da história do projeto.

4.1. Entendendo o estado dos arquivos

Sempre que você rodar o comando **git status**, o Git vai te dizer em que estado cada arquivo se encontra:

Untracked (Não rastreado): Arquivo novo que o Git ainda não conhece. Ele está na pasta, mas o Git o ignora.
Modified (Modificado): Você alterou um arquivo que já existia, mas ainda não enviou para a área de preparação (Staging).
Staged (Preparado): Aqui, o arquivo já está na Staging Area. Ele está pronto para ser salvo no próximo commit.
Committed (Consolidado): O arquivo foi salvo com sucesso no banco de dados do Git.

Que tal vermos agora como isso se aplica na prática? Por enquanto, apenas aplique os seguintes códigos fornecidos e entenda o conceito de estado dos arquivo. Os comandos de git aplicados serão explicados mais adiante na apostila. Agora, abra o seu terminal e vamos botar a mão na massa

1. Escolha um caminho, crie uma pasta para teste e entre nela:

```
#mkdir --> criação de diretório
mkdir teste-git


#cd --> permite a navegação entre pastas
cd teste-git
git init
```

2. Agora, crie um arquivo e veja ele no estado "Untracked"

```
#echo --> Escreve algo, acompanhado do ">" ele direciona aquele texto ao arquivo
echo "Olá, Git" > notas.txt
git status
```

Nota: *Note que o arquivo aparecerá em vermelho. O Git sabe que ele existe, mas não está cuidando dele.*

3. Mova o arquivo para a Staging Area:

```
git add notas.txt
git status
```

Nota: *Agora ele deve aparecer em verde. Ele está pronto para o commit.*

4. Salve a versão no Repositório Local:

```
git commit -m "Meu primeiro commit"
git status
```

Nota: *O Git dirá que a árvore de trabalho está limpa ("nothing to commit, working tree clean"). Seu arquivo agora está Committed.*

5. Faça uma alteração e veja o estado "Modified":

```
echo "Nova linha" >> notas.txt
git status
```

Nota: *O arquivo voltará a ficar* ***vermelho****, mas agora como "modified". O ciclo recomeça!*

Observações: Muitos erros acontecem porque o dev esquece de dar o git add. Lembre-se: o commit só salva o que está em verde (na Staging Area). Se o arquivo estiver vermelho, ele ficará de fora do histórico de snapshot do projeto.

5. Apresentação dos Códigos Principais

Nesta seção, vamos aprender os comandos que você usará 99% do tempo enquanto estiver programando. Lembre-se: o Git é como um GPS; se você não souber onde está, não saberá para onde ir.

5.1 git init: Criando seu primeiro repositório

Toda grande jornada começa com um ***init***. Esse comando transforma uma pasta comum em um repositório Git, criando uma pasta oculta chamada ***.git*** onde toda a "mágica" do histórico será guardada.

No terminal, dentro da pasta do seu projeto:

```
git init
```

Dica: *Se você apagar a pasta oculta .git, você apaga todo o histórico do projeto, mas os arquivos atuais continuam lá.*

5.2 git status: Verificando sua posição atual

Este é o comando que você mais vai digitar. Ele te diz quais arquivos foram modificados, quais estão prontos para o commit e o que o Git ainda não está rastreando:

```
git status
```

Dica: *Sempre rode o status antes e depois de qualquer outro comando. É a sua bússola.*

5.3 git add: Selecionando as mudanças

Lembra da **Staging Area**? O ***add*** é o comando que coloca os arquivos lá. Você pode adicionar um arquivo por vez ou tudo de uma vez:

```
# Para adicionar um arquivo específico:
git add arquivo.c

# Para adicionar TUDO o que foi alterado (o mais comum):
git add .
```

5.4 git commit: Criando pontos na história

Lembra do ***commit*** que conversamos no tópico anterior? Então, ele seria o “Save”, com ele temos a descrição do que foi alterado dentro do projeto. Ele consolida as mudanças que estavam na Staging Area. Como desenvolvedor e para ser mais profissional, evite mensagens como "ajustes", "testes" ou "projeto final feito vai na fé...". Use o padrão: **Verbo no imperativo + o que foi feito.**

Ruim: git commit -m "mudei umas coisas"

Bom: git commit -m "Cria a função de cálculo de média"

Exemplo na prática:

git commit -m "Adiciona estrutura inicial do projeto"

Dentro do mercado, utiliza-se mais do que apenas esse padrão citado. Em empresas, a especificação do tipo de alteração que você realizou é considerada também uma boa prática. Abaixo, deixaremos como recomendação um artigo referente a essas boas práticas de commit

GitHub - JuniorLima22/padroes-e-nomenclaturas-no-git: Esse projeto consta com link's de pesquisas para padrões, nomenclaturas e fluxo de trabalho com Git. · GitHub

5.5 git log: Navegando pelo passado do projeto

Quer saber quem fez o quê e quando? O ***log*** mostra toda a linha do tempo do seu repositório:

Versão completa (com autor, data e ID do commit):
git log

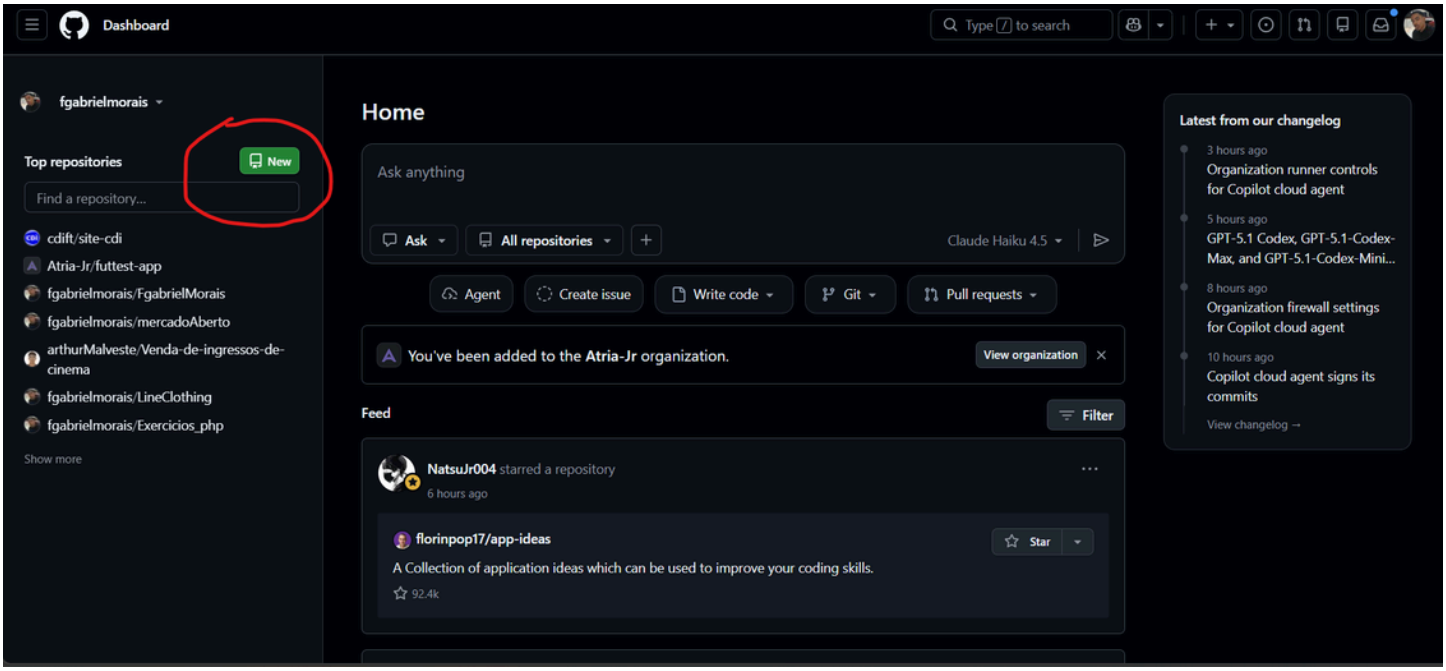
6. Conexão com o Github

Até agora, tudo o que fizemos ficou "escondido" na sua máquina. Se o seu computador pifar, você perde tudo. O GitHub entra como o nosso servidor seguro e nossa vitrine. Vamos aprender a "parear" seu projeto local com a nuvem.

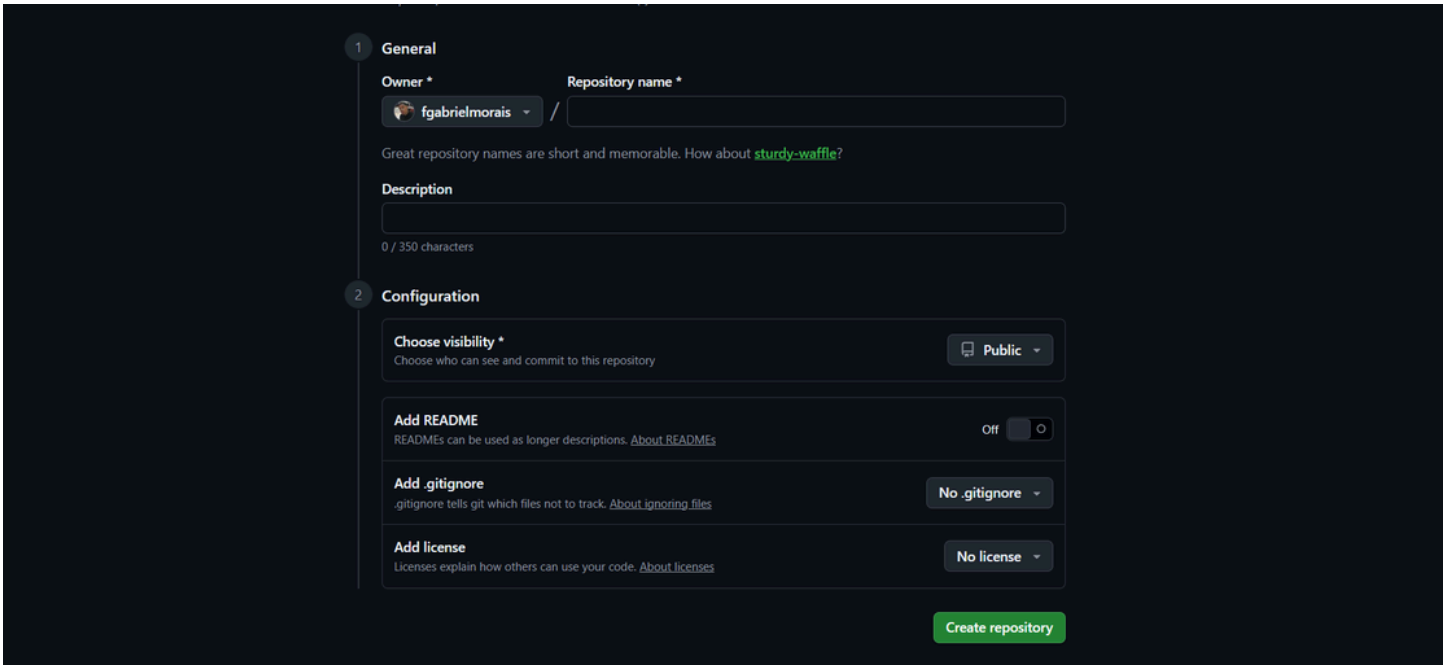
6.1. git remote: Criando a ponte

O ***remote*** é o comando que diz ao seu Git local: *"Ei, existe um servidor na internet que será o dono deste projeto"*. O nome padrão que damos a essa conexão é **origin:**

1. **Vá ao seu GitHub e crie um novo repositório (New Repository).**

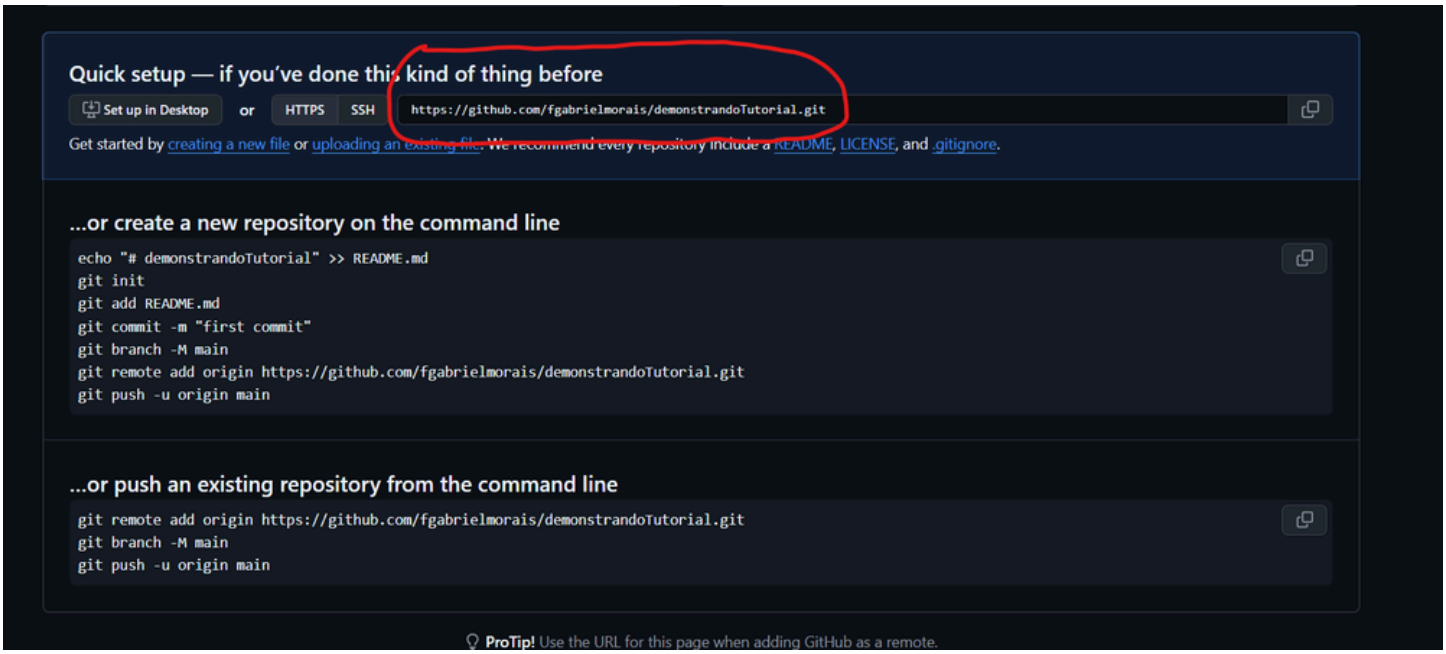


2. Há algumas caixas que você pode escolher se quer incluir no seu novo repositório ou não.



Nota: Para você não ficar perdido, o **README** seria o arquivo que colocamos parte da documentação (o que é, como inicializar etc) do projeto. O **.gitignore** é um arquivo onde restringimos que o git jogue no repositório arquivos que contenham informações importantes do projeto (.env, node_modules etc).

3. **Copie a URL (HTTPS) que o GitHub vai te mostrar.**



4. **No seu terminal, digite:**

##na mesma linha do terminal
git remote add origin https://github.com/seu-usuario/nome-do-projeto.git

6.2. git push: Enviando seu trabalho

Agora que a ponte está construída, você precisa "empurrar" seus commits para o servidor. No seu primeiro envio, você precisa dizer ao Git qual "ramo" (branch) está enviando:

git push -u origin main

Nota: *O -u serve para "lembrar" essa configuração. Nas próximas vezes, você só precisará digitar git push.*

6.3. git clone: Baixando projetos existentes

O *clone* é como você baixa um projeto inteiro do GitHub para o seu PC. Ele já vem com todo o histórico de commits e a conexão *remote* configurada. É o comando que você usará para baixar os exemplos das aulas ou contribuir em projetos de amigos:

##Vá para uma pasta vazia e digite:
git clone https://github.com/usuario/projeto-exemplo.git

Nota: *Isso criará uma pasta na sua máquina com o nome do projeto e todos os arquivos dentro.*

6.4. git pull: Sincronizando a máquina com a nuvem

Imagine que você trabalhou no laboratório da Unicamp e deu um `push`. Chegando em casa, o código no seu notebook está desatualizado. O `pull` serve para "puxar" as novidades da nuvem para o seu PC:

```
git pull origin main
```

O Momento da Autenticação

Na primeira vez que você der um `push`, o Git vai pedir seu login.

Atenção: O GitHub não aceita mais a sua senha comum do site no terminal por segurança. Você precisará de um **Personal Access Token (PAT)** ou configurar uma **Chave SSH**.

🌐 **Criando um token de acesso no github - 04**

7. Conteúdo Complementar: Branches (Ramificações)

Até agora, pensamos no Git como uma linha reta de commits. Mas e se você quiser testar uma funcionalidade nova e perigosa sem estragar a versão do código que já está funcionando? Para isso usamos as Branches. Imagine que o seu projeto é uma estrada principal (chamada **main**). Uma *Branch* é como uma saída dessa estrada para uma via paralela.

Você cria uma branch para trabalhar em algo específico (ex: um sistema de login).
Você pode errar, deletar e quebrar tudo nessa via paralela.
A estrada principal (main) continua intacta e segura.

Por que usar?

Segurança: Você nunca mexe direto na "versão de produção" (a que funciona).
Organização: Cada integrante do grupo trabalha em sua própria branch.
Experimentos: Quer testar uma biblioteca nova? Cria uma branch. Se não gostar, é só deletar a branch e nada terá acontecido com o projeto original.

7.2. Comandos: Criando e Navegando entre mundos

Para gerenciar essas realidades paralelas, usamos basicamente esses comandos:

```
# 1. Ver em qual branch você está agora:
git branch

# 2. Criar uma nova branch chamada 'feature-login':
git branch feature-login

# 3. Entrar na branch que você criou (mudar de realidade):
git checkout feature-login
```

7.3. Conceito de Merge e Pull Request

Depois que você terminou sua funcionalidade na branch paralela e testou tudo, você precisa trazer essas mudanças de volta para a estrada principal. Isso é o que chamamos Merge. Ou seja, é o ato de unir uma branch à outra. No terminal, você volta para a main e "chama" a sua branch:

```
git checkout main
git merge feature-login
```

Pull Request (PR): No mundo profissional (e no GitHub), você não dá o merge direto no seu PC. Você envia sua branch para o GitHub e abre um "Pedido de Puxada" (Pull Request). Seus colegas revisam seu código e, se estiver tudo certo, o GitHub faz o merge.

8. Se interessou no assunto?

Parabéns, bixão! Você chegou ao final da apostila e agora tem uma base sobre como funciona o Git/Github. O assunto é muito importante de ser estudado para você se desenvolver bem como dev e no ambiente de trabalho em equipe. Dada a situação, vou estar disponibilizando mais alguns vídeos/materiais que seriam interessantes você dar uma olhada para fixar melhor o conteúdo

Github para iniciantes:
🌐 **Tutorial GitHub para Iniciantes | Como usar o GitHub 2026 Atualizado**

Como enviar o seu projeto para o Github:
🌐 **COMO ENVIAR UM PROJETO PARA O GITHUB**

Semântica dos commits:
🌐 **Commits: O Bom, o Mau e o Feio – Como Evitar Erros no Git**

Branch e Merge:
🌐 **GIT BRANCH E GIT MERGE - Git e GitHub #03**

Curso de git/github do Gustavo Guanabara
🌐 **Curso de Git e GitHub: grátis, prático e sem usar comandos no terminal**

9. Algumas das abreviações e o que elas significam na hora de inseri-las no terminal:

Na página seguinte tem uma tabela com abreviações que são geralmente usadas nos códigos git

Flag	Nome Completo	Exemplo de Uso	O que ela faz?
-m	--message	git commit -m "texto"	Permite escrever a mensagem do commit direto no terminal, sem abrir um editor.
-u	--set-upstream	git push -u origin main	Vincula sua branch local à do servidor. Você só precisa usar isso no primeiro envio.
-b	branch	git checkout -b nome	Atalho que cria uma nova branch e já muda para ela ao mesmo tempo.
-d	--delete	git branch -d nome	Deleta uma branch de forma segura (só apaga se você já fez o merge dela).
-D	--delete --force	git branch -D nome	Força a exclusão de uma branch, mesmo que as mudanças não tenham sido salvas.
-v	--verbose	git remote -v	"Modo detalhado". Mostra informações extras, como as URLs dos links remotos.
-a	--all	git commit -a -m "..."	Faz o add e o commit de todos os arquivos modificados em um único passo.
.	(Diretório atual)	git add .	Não é exatamente uma flag, mas indica ao Git para incluir **tudo** da pasta atual.